ECEN 5113

HW: 8

Saira Khan

November 25, 2025

Procedure:

For this lab, a IBM RS/6000 Multiply-Add-Fused Functional Unit was created. This unit first multiplies two numbers together then adds them (complying to IEEE 754 format) in one step. Instead of splitting the multiplication and the addition into two different steps, this unit has a 2 stage pipeline that allows it to multiply and add at the same time. In order to implement this unit, first, a Verilog code was created to model its logic. Using Vivaod, a behavioral synthesis was performed to check the logic of the code with several test cases. The Cadence RTL compiler was used to generate a netlist, a textual representation of the circuit and its connections. Then Cadence Innovus was used to generate a layout and schematic. The design was then verified with a DRC and connectivity check.

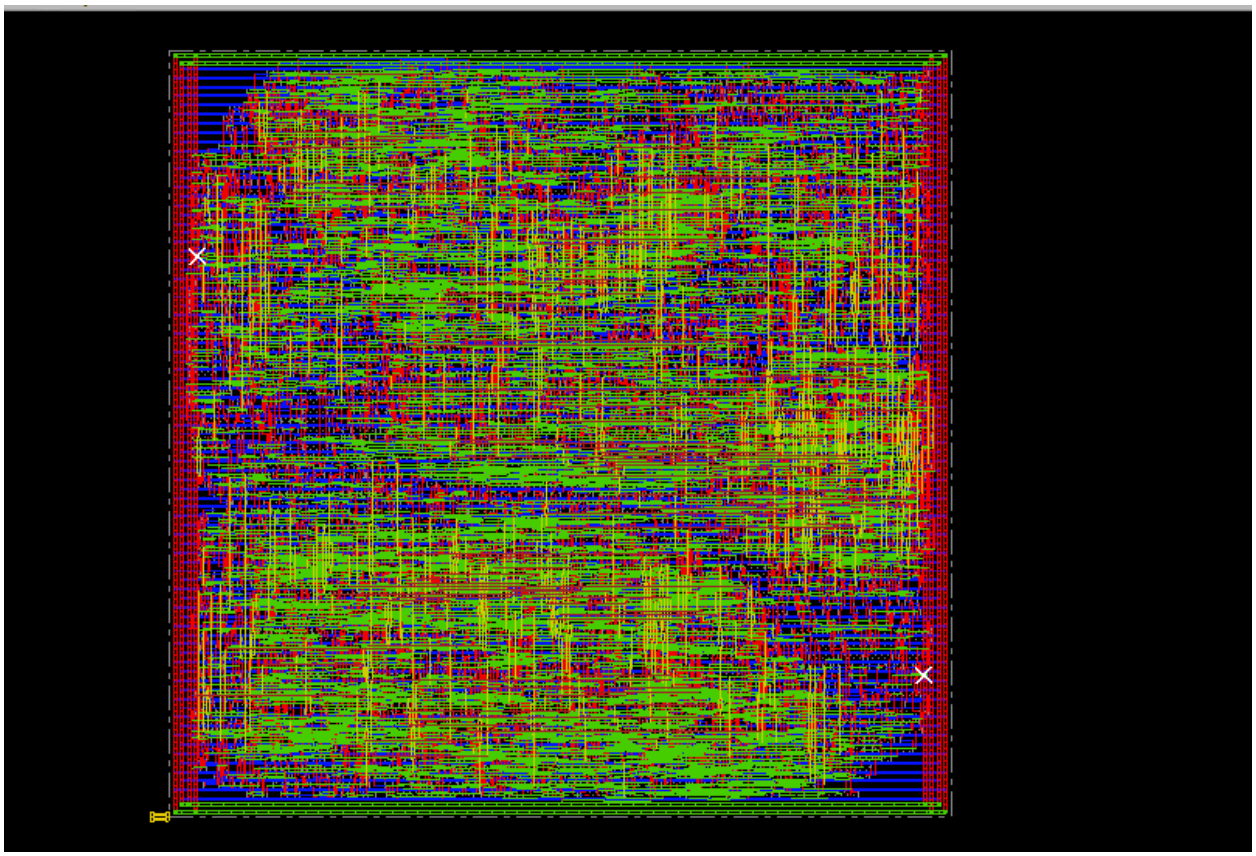

Figure 1: Layout of IBM MAF

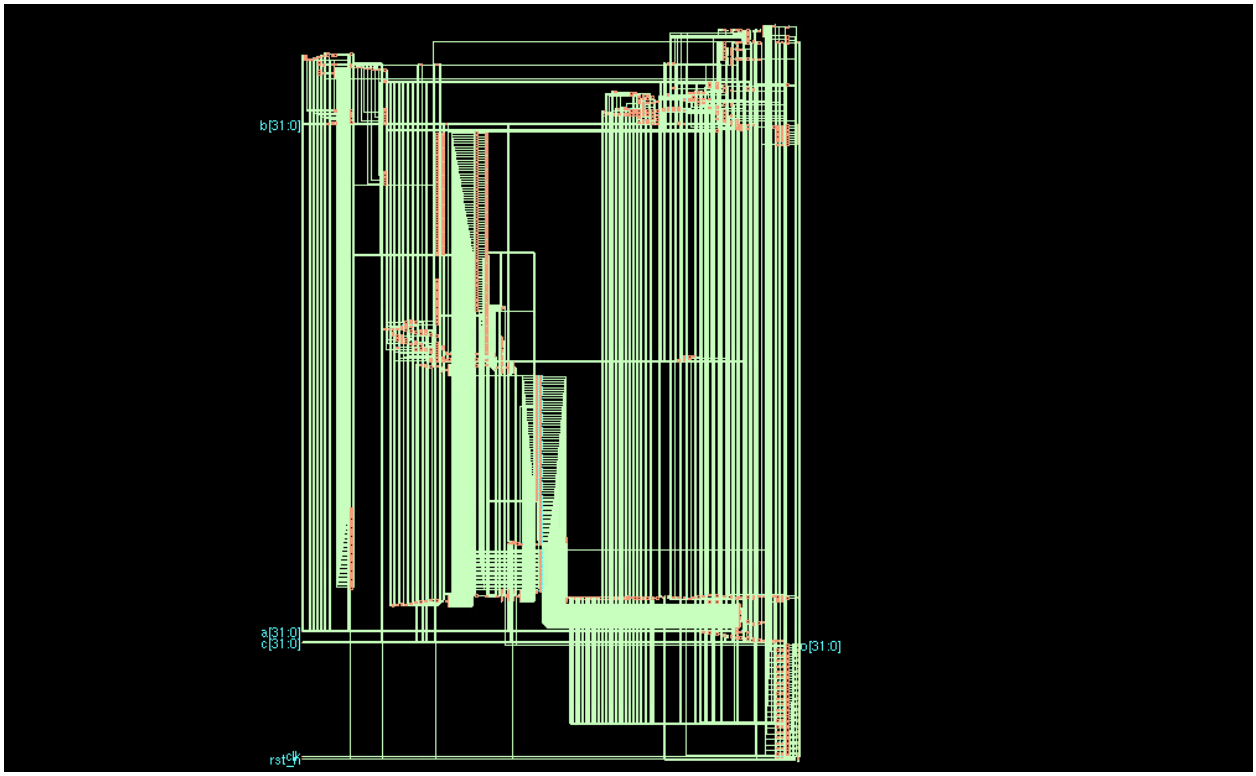

Figure 2: Schematic of IBM MAF

Observations:

There were two violations found when running a DRC check on the IEEE multiplier, but it was few enough where it could warrant ignoring. In this case, there were two shorts happening on Metal 1. Looking at the time report, the total delay is seen to be 51,882 ps (around 52ns). The two biggest delays happen when the exponents are being compared for alignment (15.3ns) and at the mantissa addition (17ns). To reduce the delay happening in the circuit, the ripple carry adder could be replaced with something like a carry lookahead adder. With a carry lookahead adder, the carries are calculated simultaneously using the original inputs (product of A*B and C). However, this is a more complex design that takes up more area and power as well. From the power report, the whole circuit consumes 7,019 mW. It shows that the multiplier is taking up 5,446 mW of dynamic power which is a big chunk. To reduce this, clock gating could be incorporated to turn off the multiplier when it is not being used. Looking at the area report, the total area of the circuit is 83,153 μm^2. The multiplier takes up most of the area.

Results:

Figure 3 shows the behavioral synthesis output from Vivado. Table 1 shows the test cases that were used and the figures show the actual output matches the intended output. The waveform demonstrates how the unit acts as a 2-stage pipeline. At 50ns, the inputs a, b, and c enter. At

60ns, the next clock cycle, multiplication happens. At 70ns, the product is added to C and the output is realized. From 50ns to 70ns shows a clock cycle of 2 (each clock cycle is 10ns). It shows overlapping execution happening as well. At 70ns, multiplication between the next inputs (5 and 2) is happening while the addition of 6 and 4 happens.

The rest of the figures show the result of creating a layout in Innovus. There are no connectivity or check placement issues that occur through the process.

Table 1: Test Cases

A	**B**	**C**	**Output**
2	3	4	10
5	2	0	10
7	1	3	10
-2	3	4	-2
0.5	0.25	0.125	0.25
2	3	-6	0
0	5	3	3
5	0	3	3
100	200	1000	21000
2	-3	10	4
1.5	2.5	0.5	4.25
NaN	2	3	NaN
infinity	2	3	infinity

```
# run 1000ns
time=60000 | a=40000000 | b=40400000 | c=40800000 |  o=41200000
time=90000 | a=40a00000 | b=40000000 | c=0 |  o=41200000
time=120000 | a=40e00000 | b=3f800000 | c=40400000 |  o=41200000
time=150000 | a=c0000000 | b=40400000 | c=40800000 |  o=c0000000
time=180000 | a=3f000000 | b=3e800000 | c=3e000000 |  o=3e800000
time=210000 | a=40000000 | b=40400000 | c=c0c00000 |  o=0
time=240000 | a=0 | b=40a00000 | c=40400000 |  o=40400000
time=270000 | a=40a00000 | b=0 | c=40400000 |  o=40400000
time=300000 | a=42c80000 | b=43480000 | c=447a0000 |  o=46a41000
time=330000 | a=40000000 | b=c0400000 | c=41200000 |  o=40800000
time=360000 | a=3fc00000 | b=40200000 | c=3f000000 |  o=40880000
time=395000 | a=40800000 | b=40a00000 | c=40c00000 |  o=41d00000
time=425000 | a=40800000 | b=40a00000 | c=40c00000 |  o=41d00000
time=455000 | a=7fc00000 | b=40000000 | c=40400000 |  o=7fc00000
time=485000 | a=7f800000 | b=40000000 | c=40400000 |  o=7f800000
```

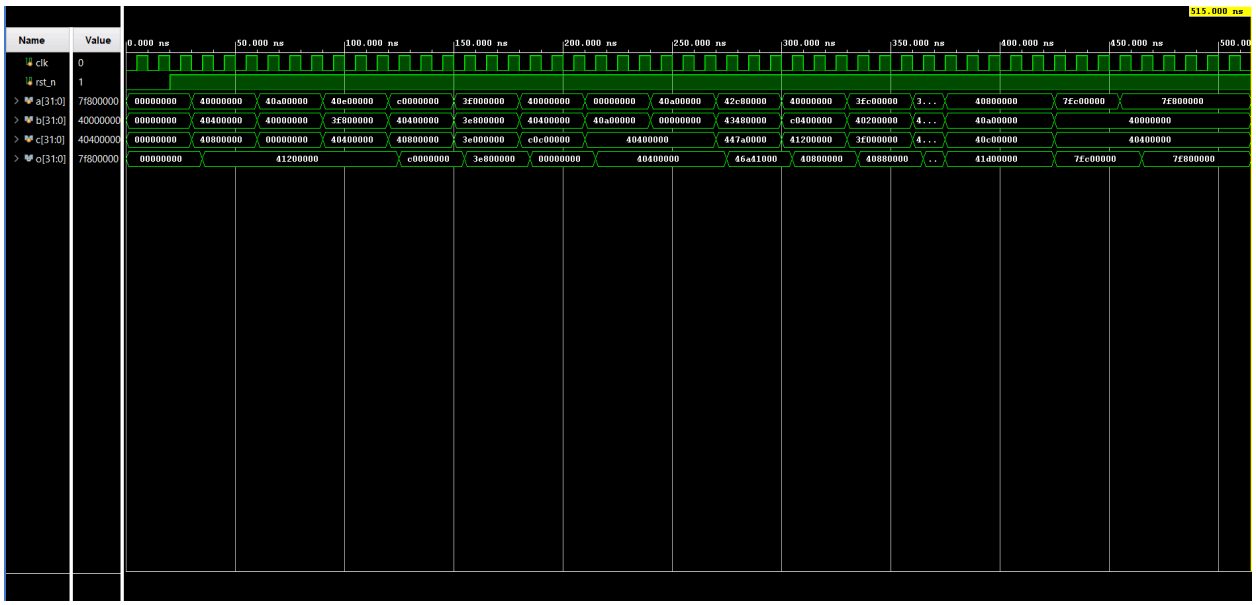

Figure 3: Vivado Output of IBM MAF

```
          Trick     Calls     Accepts   Attempts    Time(secs)
-------------------------------------------------------------
          undup         0  (        0 /        0 )  0.01
        rem_buf         0  (        0 /        0 )  0.00
        rem_inv         0  (        0 /        0 )  0.00
       merge_bi         0  (        0 /        0 )  0.00
     rem_inv_qb         0  (        0 /        0 )  0.00
       io_phase         2  (        0 /        0 )  0.01
      gate_comp       130  (        0 /        0 )  0.31
      gcomp_mog        16  (        0 /        0 )  0.19
      glob_area        50  (        0 /       50 )  0.01
      area_down        18  (       14 /       14 )  0.17


  Done mapping ibm
  Synthesis succeeded.
```

Figure 4: RC Synthesis

```
+----------------+-----------------------------+------------------+
|                |              Real           |       Total      |
|    DRVs        +-----------------+-----------+------------------|
|                |  Nr nets(terms) | Worst Vio |  Nr nets(terms)  |
+----------------+-----------------+-----------+------------------+
|   max_cap      |      0 (0)      |   0.000   |      0 (0)       |
|   max_tran     |      0 (0)      |   0.000   |      0 (0)       |
|   max_fanout   |      0 (0)      |     0     |      0 (0)       |
|   max_length   |      0 (0)      |     0     |      0 (0)       |
+----------------+-----------------+-----------+------------------+

Density: 49.980%
Routing Overflow: 0.00% H and 0.00% V
------------------------------------------------------------------
Reported timing to dir timingReports
Total CPU time: 0.84 sec
Total Real time: 2.0 sec
Total Memory Usage: 1509.144531 Mbytes
*** timeDesign #1 [finish] : cpu/real = 0:00:00.8/0:00:01.9 (0.4), totSession cpu/real = 0:01:23.
5/0:10:19.2 (0.1), mem = 1509.1M
```

```
Setup views included:
 default_view_setup

+--------------------+---------+---------+
|     Setup mode     |   all   | default |
+--------------------+---------+---------+
|           WNS (ns):|  0.000  |  0.000  |
|           TNS (ns):|  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |
|          All Paths:|    0    |    0    |
+--------------------+---------+---------+
```

Figure 5: Pre-CTS Timing Report

```
------------------------------------------------------------------

Setup views included:
 default_view_setup

+--------------------+---------+---------+
|     Setup mode     |   all   | default |
+--------------------+---------+---------+
|           WNS (ns):|  0.000  |  0.000  |
|           TNS (ns):|  0.000  |  0.000  |
|    Violating Paths:|    0    |    0    |
|          All Paths:|    0    |    0    |
+--------------------+---------+---------+
```

```
+--------------------+---------+---------+

+----------------+-------------------------------+------------------+
|                |              Real             |       Total      |
|    DRVs        +------------------+------------+------------------|
|                |  Nr nets(terms)  | Worst Vio  |  Nr nets(terms)  |
+----------------+------------------+------------+------------------+
|   max_cap      |      0 (0)       |   0.000    |      0 (0)       |
|   max_tran     |      0 (0)       |   0.000    |      0 (0)       |
|   max_fanout   |      0 (0)       |     0      |      0 (0)       |
|   max_length   |      0 (0)       |     0      |      0 (0)       |
+----------------+------------------+------------+------------------+

Density: 49.980%
Total number of glitch violations: 0
-------------------------------------------------------------------
Reported timing to dir timingReports
Total CPU time: 3.0 sec
Total Real time: 4.0 sec
Total Memory Usage: 1563.589844 Mbytes
Reset AAE Options
*** timeDesign #2 [finish] : cpu/real = 0:00:03.0/0:00:04.2 (0.7), totSession cpu/real = 0:01:54.
6/0:13:28.2 (0.1), mem = 1563.6M
```

Figure 6: Post-CTS Timing Report

```
Quick connect...
2. crosby3.utsarr.net (drb694)
/home/drb694/ibm/
Name
ibm_Net.v
ibm_pwr.rpt
ibm_time.rpt
innovus_demo.conf
mmmc.tcl
rc.cmd
rc.cmd1
rc.log
rc.log1
slow.lib
Remote monitoring
Follow terminal folder

 VERIFY DRC ...... Sub-Area: {0.000 0.000 220.480 212.160} 1 of 4
 VERIFY DRC ...... Sub-Area : 1 complete 0 Viols.
 VERIFY DRC ...... Sub-Area: {220.480 0.000 433.620 212.160} 2 of 4
 VERIFY DRC ...... Sub-Area : 2 complete 1 Viols.
 VERIFY DRC ...... Sub-Area: {0.000 212.160 220.480 423.360} 3 of 4
 VERIFY DRC ...... Sub-Area : 3 complete 1 Viols.
 VERIFY DRC ...... Sub-Area: {220.480 212.160 433.620 423.360} 4 of 4
 VERIFY DRC ...... Sub-Area : 4 complete 0 Viols.

 Verification Complete : 2 Viols.

 Violation Summary By Layer and Type:

                  Short   Totals
       Metal1        2        2
       Totals        2        2

 *** End Verify DRC (CPU: 0:00:00.7  ELAPSED TIME: 0.00  MEM: 4.0M) ***

innovus 1> VERIFY_CONNECTIVITY use new engine.

******** Start: VERIFY CONNECTIVITY ********
Start Time: Tue Nov 25 12:10:04 2025
```

```
#################################################################
#  Generated by:      Cadence Innovus 20.16-s101_1
#  OS:                Linux x86_64(Host ID crosby3.utsarr.net)
#  Generated on:      Tue Nov 25 12:09:12 2025
#  Design:            ibm
#  Command:           verify_drc
#################################################################
#set_verify_drc_mode -check_ndr_spacing auto -check_only default -check_same_via_cell false -disable_rules "" -exclude_pg_net false -ignore_cell_blockage false -ignore_trial_route false -
limit 1000 -report ibm.drc.rpt -use_min_spacing_on_block_obs auto

SHORT: ( Metal Short ) Regular Wire of Net mul_187_41/n_1321 & Blockage of Cell mul_187_41/g12783  ( Metal1 )
Bounds : ( 418.060, 78.005 ) ( 418.290, 78.235 )


SHORT: ( Metal Short ) Regular Wire of Net n_157 & Pin of Cell g12141  ( Metal1 )
Bounds : ( 15.395, 309.285 ) ( 15.640, 309.515 )


  Total Violations : 2 Viols.
```

Figure 7:DRC Check Results

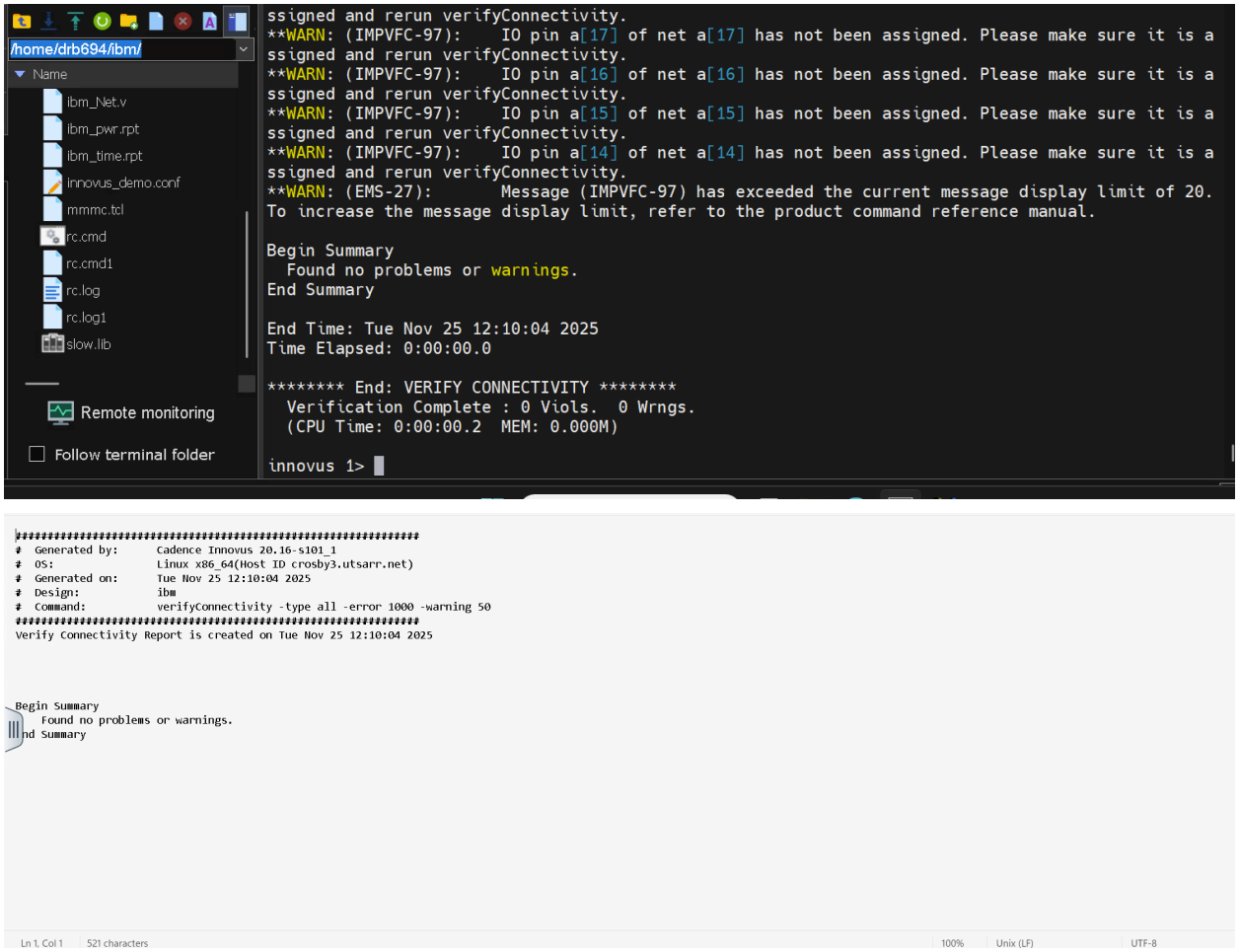

Figure 8:Connectivity Check Results

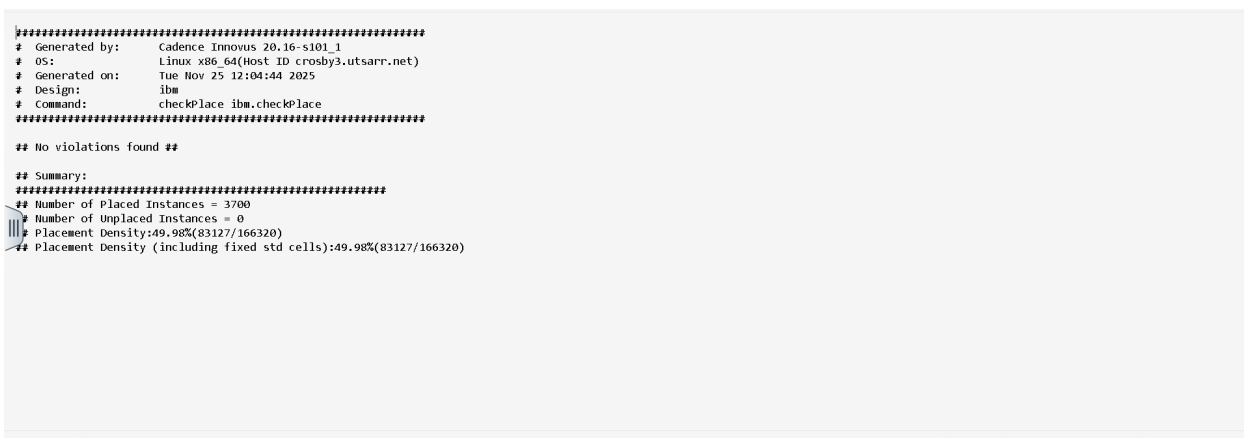

Figure 9:Check Placement Results

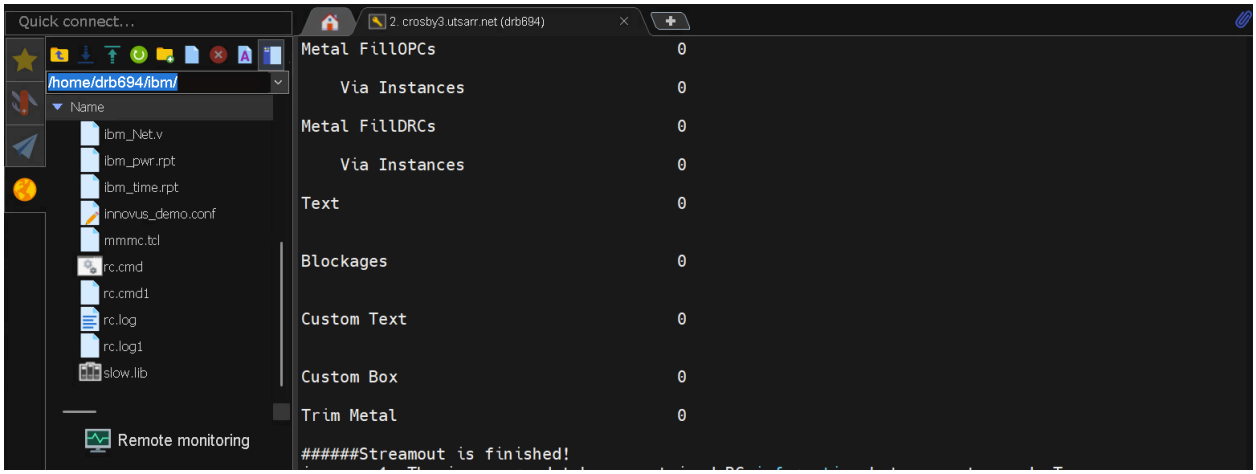

Figure 10:GDSII Output

```
============================================================
  Generated by:           Encounter(R) RTL Compiler v14.10-s022_1
  Generated on:           Nov 25 2025  11:51:37 am
  Module:                 ibm
  Technology library:     tsmc18 1.0
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

            Instance              Cells  Cell Area  Net Area  Total Area  Wireload
-----------------------------------------------------------------------------------------
ibm                                 3700      83153         0       83153    <none> (D)
  mul_187_41                        1181      30866         0       30866    <none> (D)
  srl_268_50                         276       4570         0        4570    <none> (D)
  sll_375_46                         237       3998         0        3998    <none> (D)
  csa_tree_eq_398_66_groupi          206       3899         0        3899    <none> (D)
  final_adder_mux_result_293_12       48       3320         0        3320    <none> (D)
  final_adder_mux_result_303_28       49       3300         0        3300    <none> (D)
  srl_248_43                         200       3084         0        3084    <none> (D)
  csa_tree_sub_312_38                 96       2036         0        2036    <none> (D)
  csa_tree_sub_305_41                 94       2022         0        2022    <none> (D)
  gte_303_28                         105       1553         0        1553    <none> (D)
  csa_mux_b_mux_result_303_28         97       1124         0        1124    <none> (D)
  csa_mux_a_mux_result_303_28         97       1124         0        1124    <none> (D)

 (D) = wireload is default in technology library
```

Figure 11: Area Report

```
============================================================
  Generated by:           Encounter(R) RTL Compiler v14.10-s022_1
  Generated on:           Nov 25 2025  11:51:37 am
  Module:                 ibm
  Technology library:     tsmc18 1.0
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

                                   Leakage    Dynamic      Total
          Instance          Cells Power(nW)  Power(nW)   Power(nW)
-------------------------------------------------------------------
ibm                          3700  2747.712 7016652.175 7019399.887
  mul_187_41                 1181  1152.548 5444961.003 5446113.551
  final_adde..result_293_12    48   185.541   62933.717   63119.258
  final_adde..result_303_28    49   183.519   52745.128   52928.647
  csa_tree_eq_398_66_groupi   206   148.783   68326.801   68475.584
  srl_268_50                  276    81.420  106743.809  106825.229
  csa_tree_sub_312_38          96    73.724   41139.581   41213.305
  csa_tree_sub_305_41          94    72.361   34615.152   34687.513
  sll_375_46                  237    63.310   51298.143   51361.452
  srl_248_43                  200    56.434  114239.854  114296.288
  gte_303_28                  105    40.913    9981.707   10022.620
  csa_mux_b_..result_303_28    97    19.748   26507.973   26527.721
  csa_mux_a_..result_303_28    97    19.705   15871.360   15891.065
  csa_tree_sub_376_49           0     0.000       0.000       0.000
```

Figure 12: Power Report

```
============================================================
  Generated by:           Encounter(R) RTL Compiler v14.10-s022_1
  Generated on:           Nov 25 2025  11:51:37 am
  Module:                 ibm
  Technology library:     tsmc18 1.0
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

          Pin               Type      Fanout  Load Slew Delay Arrival
                                              (fF) (ps)  (ps)   (ps)
-------------------------------------------------------------------------
mult_man_prod_reg[47]/CK                                 0             0 R
mult_man_prod_reg[47]/Q    DFFRX1         51 155.2 1915 +1679    1679 R
g8418/B                                                        +0    1679
g8418/Y                    NAND2XL         3   7.8  343  +173    1852 F
g8411/AN                                                       +0    1852
g8411/Y                    NAND2BXL        3   7.8  149  +274    2125 F
g8402/B0                                                       +0    2125
g8402/Y                    OAI2BB2XL       4  12.1  464  +316    2441 R
g8393/A0N                                                      +0    2441
g8393/Y                    AOI2BB2XL       2   8.5  349  +276    2718 R
g8378/B                                                        +0    2718
g8378/CO                   ADDHXL          2   9.4  210  +257    2975 R
g8377/A                                                        +0    2975
g8377/Y                    INVXL           1   3.4   75   +64    3039 F
g8372/A0                                                       +0    3039
```

```
g1401/A0N                                                      +0   12800
g1401/Y                    AOI2BB1XL       1   3.2  178  +218   13018 R
g1400/B0                                                       +0   13018
g1400/Y                    AOI21XL         2   5.0  155   +84   13102 F
g1399/A1N                                                      +0   13102
g1399/Y                    OAI2BB1XL       1   3.0  100  +204   13306 F
g1398/B0                                                       +0   13306
g1398/Y                    OAI21XL         2   5.4  289  +121   13427 R
g1397/A                                                        +0   13427
g1397/Y                    NOR2XL          1   3.5  106   +81   13509 F
g1396/B0                                                       +0   13509
g1396/Y                    OAI2BB2XL       2   5.4  292  +207   13716 R
g1395/A                                                        +0   13716
g1395/Y                    NOR2XL          1   3.5  104   +82   13798 F
g1394/B0                                                       +0   13798
g1394/Y                    OAI2BB2XL       2   5.6  298  +210   14008 R
g1393/A0N                                                      +0   14008
g1393/Y                    AOI2BB1XL       1   3.2  178  +218   14225 R
g1392/B0                                                       +0   14225
g1392/Y                    AOI21XL         1   3.3  136   +73   14298 F
g1391/A0                                                       +0   14298
g1391/Y                    OAI22XL         2   5.6  381  +184   14482 R
g1390/A0N                                                      +0   14482
g1390/Y                    AOI2BB1XL       1   3.2  178  +226   14708 R
g1389/B0                                                       +0   14708
g1389/Y                    AOI21XL         2   5.0  167   +84   14792 F
g1388/A1N                                                      +0   14792
g1388/Y                    OAI2BB1XL       1   3.0  100  +207   14999 F
g1387/B0                                                       +0   14999
```

```
g12267/Y                   NAND2BXL        2   7.0  234  +231   48814 R
g12213/C0                                                      +0   48814
g12213/Y                   AOI221XL        1   2.6  379   +86   48900 F
g12165/A                                                       +0   48900
g12165/Y                   MXI2XL          3   9.5  568  +312   49212 R
g12156/B                                                       +0   49212
g12156/Y                   NOR4BXL         1   2.3  186  +145   49357 F
g12141/AN                                                      +0   49357
g12141/Y                   NOR4BXL         1   2.8  212  +229   49587 F
g12137/B                                                       +0   49587
g12137/Y                   NAND2XL         1   3.0  156  +130   49717 R
g12136/B0                                                      +0   49717
g12136/Y                   OAI21XL         1   3.5  132  +110   49827 F
g12135/B0                                                      +0   49827
g12135/Y                   AOI211XL        2   7.6  451  +285   50112 R
g12134/B0                                                      +0   50112
g12134/Y                   OAI21XL         1   3.2  163  +145   50257 F
g12133/A                                                       +0   50257
g12133/Y                   NOR2XL          8  26.7  778  +481   50737 R
g12128/B                                                       +0   50737
g12128/Y                   NAND2XL        23  81.9  998  +719   51456 F
g12091/B0                                                      +0   51456
g12091/Y                   OAI221XL        1   0.0  411  +425   51882 R
o[0]                       out port                       +0   51882 R
-------------------------------------------------------------------------
Timing slack :  UNCONSTRAINED
Start-point  : mult_man_prod_reg[47]/CK
End-point    : o[0]
```

Figure 13: Timing Report

Code:

```
//ibm rs/6000 multiply-add-fused functional unit
```

```
//inputs A, B, C
//(AxB)+ C
// (Ax1) + C

//floating point unit

//fuses - performing operations at once to avoid intermediate rounding errors

//define module
module ibm(
    input wire clk,
    input wire rst_n,
    input wire [31:0] a, //input
    input wire [31:0] b, //input
    input wire [31:0] c, //input
    output reg [31:0] o //output
    );

    wire sign_a = a[31]; //sign of a
    wire sign_b = b[31]; // sign of b - 1 bit
    wire sign_c = c[31]; //sign of c
    wire [7:0] exp_a = a[30:23]; //exp a
    wire [7:0] exp_b = b[30:23]; //exponent b
    wire [7:0] exp_c = c[30:23]; //exponet c
    wire [22:0] man_a = a[22:0]; //mantissas
    wire [22:0] man_b = b[22:0];
    wire [22:0] man_c = c[22:0];


    //sign product
    wire pro_sign = sign_a ^ sign_b; //xor

    //special cases
    //A or B = 0, C is +/- inf
    //A or B +/- inf, C = 0...

    wire a_zero = (exp_a==8'd0) && (man_a==23'd0); //E and M = 0
    wire b_zero = (exp_b==8'd0) && (man_b==23'd0);
    wire c_zero = (exp_c==8'd0) && (man_c==23'd0);
    wire a_inf = (exp_a==8'hFF) && (man_a==23'd0); //E = 255 (128) M = 0
```

```
wire b_inf = (exp_b==8'hFF) && (man_b==23'd0); //infinity =N

wire c_inf = (exp_c==8'hFF) && (man_c==23'd0); //infinity =N

wire a_nan = (exp_a==8'hFF) && (man_a!=23'd0); //E = 255 M !=0

wire b_nan = (exp_b==8'hFF) && (man_b!=23'd0); //NaN - not a numbers

wire c_nan = (exp_c==8'hFF) && (man_c!=23'd0); //NaN - not a numbers



//normalized mantissas

wire [23:0] norm_a = (exp_a == 8'd0) ? {1'b0, man_a} : {1'b1, man_a};

wire [23:0] norm_b = (exp_b == 8'd0) ? {1'b0, man_b} : {1'b1, man_b};

wire [23:0] norm_c = (exp_c == 8'd0) ? {1'b0, man_c} : {1'b1, man_c};



//special handling

//NaN -- NaN

// A or B is inf and other = 0 -- invalid (0*inf)

//A*B = inf, C = finite -- inf



reg [31:0] spec_final;

reg spec;

always @(*)

    begin


    spec_final = 32'd0;

    spec = 1'b0;


    if (a_nan || b_nan || c_nan)

        begin

            spec_final = 32'h7FC00000; // E= 255 NaN

            spec = 1'b1;

        end

    else if ((a_inf && b_zero) || (b_inf && a_zero))

        begin

            spec_final = 32'h7FC00000; //nan = inf* 0

            spec = 1'b1;

        end

    else if ((a_inf || b_inf) && c_inf)

        begin

        if (pro_sign != sign_c)
```

```
        begin
            spec_final = 32'h7FC00000; // inf - inf = NaN
            spec = 1'b1;
        end
    else
        begin
            spec_final = {pro_sign, 8'hFF, 23'd0}; // inf + inf = inf
            spec = 1'b1;
        end
end
else if (a_inf || b_inf)
    begin
        spec_final = {pro_sign, 8'hFF, 23'd0}; //infinity
        spec = 1'b1;
    end
else if (c_inf)
    begin
        spec_final = {sign_c, 8'hFF, 23'd0}; //C is infinity
        spec = 1'b1;
    end
else if (a_zero || b_zero)
    begin
        if (c_zero)
            begin
                spec_final={pro_sign, 8'd0, 23'd0}; //zero
                spec = 1'b1;
            end
        else
            begin
                spec_final = c; //return C
                spec = 1'b1;
            end
 end
else
    begin
        spec_final = 32'd0;
        spec = 1'b0;
    end
end
```

```
//normalized cases

//2 stage pipeline design

//1st stages does multiplication

//2nd stage does addition

//working at the same time



//multiplication stage - stage 1

//registers to store results from multiplication stage

reg [47:0] mult_man_prod; //AxB (24 bit x 24 bit = 48 bit)

reg [8:0] mult_exp; //exponent exp_a + exp_b -127 (extra bit for overflow)

reg first_pro_sign; //sign of product A*B

reg [23:0] mant_c; //mantissa c

reg [7:0] first_exp_c; //exponent C

reg first_sign_c; //sign of c



reg first_spec; //special case flag

reg [31:0] first_spec_final;



//run on every rising edge of clock or if reset goes low

always @(posedge clk or negedge rst_n)

    begin

    if (!rst_n)

        begin

            //when reset then clear all register (set to 0)

            mult_man_prod <= 48'd0;

            mult_exp <= 9'd0;

            first_pro_sign <= 1'b0;

            mant_c <= 24'd0;

            first_exp_c <= 8'd0;

            first_sign_c <= 1'b0;

            first_spec <= 1'b0;

            first_spec_final <= 32'd0;

        end

    else

        begin
```

```
            //multiply mantissas together
            mult_man_prod <= norm_a * norm_b;



            //add exponents and sub 127
            mult_exp <= exp_a + exp_b - 8'd127;

            //sign needs to pass through
            first_pro_sign <= pro_sign;

            //pass through C for next stage
            mant_c <= norm_c;
            first_exp_c <= exp_c;
            first_sign_c <= sign_c;

            //pass through special case
            first_spec <= spec;
            first_spec_final <= spec_final;
        end
end



//addition - stage 2 !
//normalize product -- if MSB = 1 then already normalized
//if MSB not 1 then needs to be shifted left
wire mult_man_prod_norm = mult_man_prod[47];
//if not normalized shift left by 1
wire [47:0] mult_man_prod_shift = mult_man_prod_norm ? mult_man_prod : (mult_man_prod << 1);

//if mantissa is shifted left then need to subtract 1 from exponent
//shifting left is similar to dividing by 2 so need to decrease exponent
wire [8:0] exp_prod = mult_man_prod_norm ? (mult_exp + 9'd1) : mult_exp;
//align C w product to add the numbers
//exponents have to match

//calculate difference between products exponent and c exponent
//to find out how much to shift one of them by
//extend c exponent to 9 bits to match size for subtraction
wire signed [9:0] diff_exp = $signed({1'b0, exp_prod}) - $signed({2'b0, first_exp_c});

//check which number has the larger exponent
```

```
wire larger_exp = (exp_prod >= {1'b0, first_exp_c});



//align
reg [47:0] align_prod;
reg [47:0] align_c;
reg [8:0] align_exp;

//alignment logic

always @(*)
    begin
    //product is larger
    //product has bigger exp. so unchanged -- keep larger exp
        if (diff_exp >= 0)
            begin
            align_prod = mult_man_prod_shift;
            if (diff_exp < 48)
            //shift c right by diff_exp
            //24 bit mantissa gets 24 0s added to right = 48 bits
            //shift right to align w product exponent
            align_c = {mant_c, 24'd0} >> diff_exp;
            //if diff_exp > 48 - set C = 0

        else
            align_c = 48'd0;
            align_exp = exp_prod;
        end
        else
            begin
            //if bit 8 of diff_exp is 1
            //diff_exp = negative
            //diff_exp = exp_prod - exp_c
            //c = larger

            //c larger so it is unchanged
            //gets padded w zeroes to make 48 bits
            align_c = {mant_c, 24'd0};
            //shift product right by the diff_exp

            if(-diff_exp < 48)
```

```
            align_prod = mult_man_prod_shift >> (-diff_exp);
            //shift > 48 then product becomes zero bc it too small to matter
            else
            align_prod = 48'd0;
            //use c exponent as final exponent but extend to 9 bits
            align_exp = {1'b0, first_exp_c};
    end
end



//add or subtract mantissas
//check if product and c have the same sign
//if 1 then signs match --- add
//if 0 signs dont match -- subtract
wire add_sub = (first_pro_sign == first_sign_c);

//store result - extra bit for overflow
reg [48:0] result;
//store sign of final result
reg sign_result;



always @(*)
    begin
    //same signs - add
    if(add_sub)
        begin
            result = align_prod + align_c;
            sign_result = first_pro_sign;
        end
    else
    //subtract
     begin
    //if product magnitude is larger - subtract c from product
    //use product sign
        if (align_prod >= align_c)
            begin
                result = align_prod - align_c;
                sign_result = first_pro_sign;
            end
        else
```

```
            //if c is larger, subtract product from C
            //use c sign
            begin
                result = align_c - align_prod;
                sign_result = first_sign_c;
            end
    end
end

//normalize - ieee 754 format
//use function for leading zero count instead of loop
function [5:0] count_lead_zeros;
    input [48:0] value;
    integer i;
        begin
            count_lead_zeros = 49;
            for (i = 48; i >= 0; i = i - 1) begin
            if (value[i]) begin
            count_lead_zeros = 48 - i;
            i = -1; //break out of loop
        end
    end
    end
endfunction

reg [5:0] lead_zeroes; //count how many leading zeroes are there
reg [47:0] normalized_result; //mantissa
reg [8:0] final_exp; //exponent

always @(*)
    begin
    //initialize
    //assume no normalization needed first
    lead_zeroes = 0;
    normalized_result = result[47:0];
    final_exp = align_exp;

    //if bit 48 is 1 -- have overflow
    //need to shift right by 1 and increment exponent
    if(result[48]) //overflow
    begin
```

```
    normalized_result = result[48:1];

    final_exp = align_exp + 9'd1;

end


else if (result == 49'd0) //result is zero

    begin

        normalized_result = 48'd0;

        final_exp = 9'd0;

    end


else if (result[47] == 1'b0) //underflow - needs normalization

    begin

    //count leading zeros using function

    lead_zeroes = count_lead_zeros(result);


    //shift left by number of leading zeroes

    //decrease exponent

    //if no leading 0s then already normalized (>0)

    //if all lead zeroes (<48) result is 0

if(lead_zeroes > 0 && lead_zeroes < 48)

    begin

        normalized_result = result[47:0] << (lead_zeroes - 6'd1);

       final_exp = align_exp - (lead_zeroes - 9'd1);

    end

else if (lead_zeroes == 48)

    begin

        normalized_result = 48'd0;

        final_exp = 9'd0;

    end

end

end



//take bit 46-24 from normalized result

//only need 23 bits for mantissa

wire [22:0] final_mant = normalized_result[47] ? normalized_result[46:24] : normalized_result[45:23];





//detects if number is too small
```

```
//bit 8 set (negative)
// exponent is 0

wire underflow = final_exp[8] || (final_exp == 0) || (result == 49'd0);

//detects if number is too large exp > 255
//number becomes inf
wire overflow = (final_exp >= 9'd255);



//checks if exponent is valid 0-255 for 8 bit
//final[8] - bit 8 is set - neg -- underflow (if bit 8 = 1 then number is negative)
//final_exp > 255 -- too large -- overflow
//set exp to 0
//else take lower 8 bits of it


wire [7:0] final_exponent = overflow ? 8'hFF : underflow ? 8'd0: final_exp[7:0];


//final 32-bit IEEE 754
reg [31:0] norm_result;

always @(*)
   begin
      //underflow case -- zero
      //creates a signed zero
      if(underflow || (result == 49'd0))
         begin
         norm_result = {sign_result, 31'd0};
         end

      //signed infinity
      else if(overflow)
         begin
         norm_result = {sign_result, 8'hFF, 23'd0};
         end


      //normal number
```

```
            else
                begin
                norm_result = {sign_result, final_exponent, final_mant};
                end
        end

    //output the final result
    //combinational output for 2-stage pipeline

    always @(*)
        begin
        //if first stage detects a special case use that
        if(first_spec)
            o = first_spec_final;
        //else use normal result
        else
            o = norm_result;
        end

endmodule
//testbench

//define module
module ibm_tb;

    //inputs/outputs
    reg clk;
    reg rst_n;
    reg [31:0] a;
    reg [31:0] b;
    reg [31:0] c;
    wire [31:0] o;

    //instantiate module
    ibm d(clk, rst_n, a, b, c, o);



    //clock generation
    //10 ns period
    initial
        begin
```

```
        clk = 0;
        forever #5 clk = ~clk;
    end


//test cases
initial
    begin
        //initialize
        rst_n = 0;
        a = 32'h00000000;
        b = 32'h00000000;
        c = 32'h00000000;


        //reset pulse
        #20;
        rst_n = 1;

        #10;


        //case 1 - 2*3 + 4
        a = 32'h40000000; //2
        b = 32'h40400000; //3
        c = 32'h40800000; //4
        #30; //wait 2 cycles for pipeline
        $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
         $time, a, b, c, o);


        //case 2 multiply only - 5*2 + 0
        a = 32'h40A00000; //5
        b = 32'h40000000; //2
        c = 32'h00000000; //0
        #30; //wait 2 cycles for pipeline
        $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
         $time, a, b, c, o);
```

```
//add only 7*1 + 3

a = 32'h40E00000; //7

b = 32'h3F800000; //1

c = 32'h40400000; //3

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//negative #'s -2 * 3 + 4

a = 32'hC0000000; //-2

b = 32'h40400000; //3

c = 32'h40800000; //4

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//small # .5 * .25 + .125

a = 32'h3F000000; //.5

b = 32'h3E800000; //.25

c = 32'h3E000000; //.125

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//zero

a = 32'h40000000; //2

b = 32'h40400000; //3

c = 32'hC0C00000; //-6

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//zero = A

a = 32'h00000000; //0

b = 32'h40A00000; //5
```

```
c = 32'h40400000; //3

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//b = 0

a = 32'h40A00000; //5

b = 32'h00000000; //0

c = 32'h40400000; //3

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//large #s

a = 32'h42C80000; //100

b = 32'h43480000; //200

c = 32'h447A0000; //1000

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//mixed signs

a = 32'h40000000; //2

b = 32'hC0400000; //-3

c = 32'h41200000; //10

#30; //wait 2 cycles for pipeline

$display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",

 $time, a, b, c, o);



//fraction

a = 32'h3FC00000; //1.5

b = 32'h40200000; //2.5

c = 32'h3F000000; //0.5

#30; //wait 2 cycles for pipeline
```

```
    $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
     $time, a, b, c, o);



    //pipeline test -- back to back
    a = 32'h3F800000; //1
    b = 32'h40000000; //2
    c = 32'h40400000; //3
    #15; //wait 1 cycle
    a = 32'h40800000; //4
    b = 32'h40A00000; //5
    c = 32'h40C00000; //6
    #20; //wait 1 cycle
    $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
     $time, a, b, c, o);
    #30 //wati another cycle for 2nd result
    $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
     $time, a, b, c, o);

    //NAN test
    a = 32'h7FC00000; //NaN
    b = 32'h40000000; //2
    c = 32'h40400000; //3
    #30; //wait 2 cycles for pipeline
    $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
     $time, a, b, c, o);

    //infinity
    a = 32'h7F800000; //inf
    b = 32'h40000000; //2
    c = 32'h40400000; //3
    #30; //wait 2 cycles for pipeline
    $display("time=%0t | a=%0h | b=%0h | c=%0h |  o=%0h",
     $time, a, b, c, o);

    #30; //wait some more
    $finish;

end
```

```
endmodule
```